

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5 Total Marks:50

Duration: 1 Hour

Annexure A

Scenario Based Assignment

Code of Conduct:

I_SOWMIYA. D (192511063) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sowmiya. D

Annexure A

Scenario Based Assignment

1. **Scenario** : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering realtime constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees

- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes wellreasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1

i. Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path with the smallest heuristic value (straight-line distance to the destination). **Formula:**

$$f(n) = h(n)$$

Strengths:

- Makes quick decisions.
- Requires less memory.
- Suitable for emergency situations.

Weaknesses:

- Ignores actual travel cost.
- May choose flooded or unsafe routes.
- Does not always find the shortest path.

ii. A Search*

A* Search considers both the actual travel cost and the estimated distance to the goal.

Formula:

$$f(n) = g(n) + h(n) \text{ Where:}$$

- $g(n)$ = Actual cost from the start.
- $h(n)$ = Estimated cost to the goal.

Advantages:

- Finds the optimal route.
- Considers travel cost and safety.
- Adapts better to changing road conditions.

Disadvantages:

- Uses more memory.
- Takes more computation than Greedy Search.

iii. Impact of Heuristic Accuracy

- **Accurate heuristic:** Faster search and better route selection. □ **Inaccurate heuristic:** May lead to longer or unsafe paths.

Heuristic Greedy Search A* Search

Accurate Very fast

Fast and optimal

Inaccurate Poor performance Better than Greedy because it also uses actual cost

iv. Effect of Dynamic Changes (Blocked Paths)

- **Greedy Search:** Must replan if a selected path becomes blocked.
- *A Search*.* Updates path costs and finds a better alternative route.

Dynamic changes increase computation and may delay delivery, but A* handles them more effectively.

v. Recommended Algorithm Recommended: A Search* Reasons:

- Finds the shortest and safest path.
- Considers both travel cost and heuristic.
- Adapts to blocked roads and weather changes.
- More reliable for emergency medicine delivery.

Conclusion

Greedy Best-First Search is faster but may choose unsafe or non-optimal routes. A* Search balances speed and path cost, making it the best choice for real-time emergency drone navigation in flood-affected areas.

Question 2

i. Formulate the problem as an optimization problem and explain why traditional search is inefficient

The AI traffic signal system aims to minimize total waiting time, fuel consumption, and traffic congestion while adjusting signal timings continuously. This is an optimization problem because the system must find the best signal timing from many possible combinations.

Traditional search methods are inefficient because the search space is very large, traffic conditions change frequently, and exploring every possible solution takes too much time.

ii. Explain how Hill Climbing would work in this scenario and identify its limitations

Hill Climbing starts with an initial traffic signal timing and makes small changes to improve traffic flow. It accepts changes only if they produce a better solution.

Limitations:

- May get stuck at a local maximum.
- Cannot always find the best overall solution.
- Performs poorly when traffic patterns change suddenly.

iii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

Local Maxima: The algorithm finds a good traffic signal setting but misses a better one because it cannot move to a worse state temporarily.

Plateaus: Many signal timings produce the same performance, so the algorithm cannot decide which direction to move.

Ridges: The best solution requires several coordinated changes, but Hill Climbing changes only one step at a time, making progress difficult.

iv. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

Simulated Annealing occasionally accepts worse solutions, helping it escape local maxima and find better solutions later.

Genetic Algorithm creates multiple traffic signal solutions, combines the best ones, and improves them over several generations. This helps explore a larger search space and produces better results.

v. Recommend the best approach and justify your choice based on scalability and adaptability

The **Genetic Algorithm** is the best choice because it can handle large and complex traffic networks, adapts to changing traffic conditions, and finds high-quality solutions without getting stuck easily. It is more scalable and suitable for real-time smart city traffic optimization.

Conclusion

Hill Climbing is simple and fast but may get trapped in local maxima. Simulated Annealing improves exploration by accepting occasional worse solutions. The Genetic Algorithm provides the best balance of scalability, adaptability, and optimization, making it the most suitable approach for intelligent traffic signal management.

Question 3

i. Explain why this problem requires an online search agent instead of offline search The Mars rover operates in an unknown environment with no complete map. It receives information only while exploring, so it must make decisions in real time. Therefore, an **online search agent** is more suitable than an offline search agent. **ii. Analyze the challenges of partial observability and dynamic environments**

The rover has limited sensors, so it cannot detect all obstacles in advance. Hazards such as rocks and craters may appear unexpectedly. Since the environment changes continuously, the rover must update its decisions to ensure safe navigation.

iii. Describe how the rover builds and updates its internal model

The rover collects information using its sensors while moving. It stores this information in its internal model, updates the map whenever new obstacles are detected, and uses the updated model to choose safer paths.

iv. Compare online search with uninformed and informed search strategies Online Search:

Makes decisions while exploring and adapts to new information.

Uninformed Search: Searches without heuristic information and explores many unnecessary paths.

Informed Search: Uses heuristic information to guide the search but usually assumes the environment is already known.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety The

Online Search Agent is the most suitable approach because it can make real-time decisions, adapt to unknown and changing environments, avoid hazards, and safely reach its destination despite limited information.

Conclusion

An online search agent is the best choice for a Mars rover because it can explore unknown terrain, continuously update its knowledge, adapt to changing conditions, and ensure safe and efficient navigation.

Question 4

Formulate the problem as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) consists of **variables, domains, and constraints**. In the exam timetable system:

- **Variables:** Exams for each subject.
- **Domains:** Available dates, time slots, and exam halls.
- **Constraints:** No overlapping exams for students, limited exam halls, faculty availability, subject order, and special accommodations.

i. Clearly define variables, domains, and constraints **Variables:**

Subjects to be scheduled.

Domains: Available time slots, dates, and exam halls.

Constraints:

- No student should have two exams at the same time.
- Exam hall capacity should not be exceeded.
- Faculty must be available during the exam.
- Some subjects must be conducted before others.
- Special students should receive required accommodations.

ii. Explain how backtracking search works in this scenario

Backtracking assigns a time slot to each exam one by one. If a constraint is violated, it goes back to the previous assignment and tries another available option until a valid timetable is found. **iii.**

Discuss how constraint propagation (Forward Checking) improves efficiency

Forward Checking removes invalid time slots for remaining exams whenever a new assignment is made. This detects conflicts early, reduces unnecessary searches, and speeds up timetable generation. **iv. Analyze real-world challenges and justify the best strategy for scalability** Real-world challenges include a large number of students, limited exam halls, faculty availability, and increasing courses. As the problem grows, finding a valid timetable becomes more complex.

The best strategy is **Backtracking with Forward Checking** because it reduces conflicts early, improves efficiency, and generates valid timetables faster for large universities. **Conclusion**

A university exam timetable can be effectively modeled as a **Constraint Satisfaction Problem (CSP)**. Using **Backtracking Search with Forward Checking** helps satisfy all constraints while reducing computation time, making it the most suitable approach for scalable and efficient timetable scheduling.

Question 5

i. Explain how the Minimax algorithm models this problem

The **Minimax algorithm** is used in two-player games where one player tries to maximize the score while the opponent tries to minimize it. The AI evaluates all possible moves and selects the move that gives the best outcome while assuming the opponent plays optimally. **ii. Analyze the computational challenges of large game trees**

In real-time strategy games, each move creates many possible future moves, resulting in a very large game tree. Exploring every possible move requires high computation time and memory, making it difficult to make quick decisions.

iii. Explain how Alpha-Beta Pruning improves efficiency

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision. This reduces the number of nodes evaluated, saves computation time, and allows the AI to make faster decisions without changing the final result. **iv. Design an evaluation function and justify the factors considered**

The evaluation function assigns a score to each game state based on important factors such as:

- Number of remaining units
- Health of units
- Resources collected
- Map control

- Distance to the objective

A higher score indicates a better position for the AI, helping it choose the most effective move. v.

Recommend a strategy for balancing optimality and real-time performance

The best strategy is to use **Minimax with Alpha-Beta Pruning**. This approach provides nearoptimal decisions while reducing computation time, making it suitable for real-time strategy games with strict time limits.

Conclusion

Minimax helps the AI choose the best move by considering the opponent's actions. Alpha-Beta Pruning increases efficiency by removing unnecessary branches, enabling faster decisionmaking. Therefore, **Minimax with Alpha-Beta Pruning** is the most suitable approach for developing an intelligent and efficient game AI.